

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

Arion Interpreter

Lampiran - Milestone 4 & 5

Guidebook Konvensi Intermediate Code

Lampiran Milestone 4

Guidebook Konvensi

Intermediate Code

Dokumen ini merupakan panduan teknis implementasi Intermediate Code (IC) untuk proyek TBFO. Intermediate Code ini dirancang berbasis Stack Machine dengan mengadopsi struktur konvensi ORKOM untuk manajemen memori dan eksekusi instruksi.

Daftar Revisi

[DD-MM-YY] Revisi X. Isi Revisi

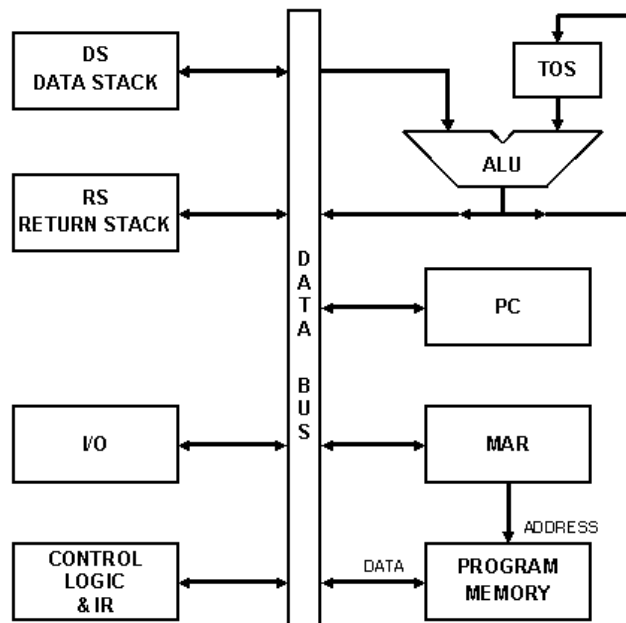
Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Model Eksekusi Stack.....	5
II. Katalog Instruksi Intermediate Code.....	7
III. Skema Validasi Runtime.....	9
IV. Contoh Penulisan Kode.....	10

I. Model Eksekusi Stack

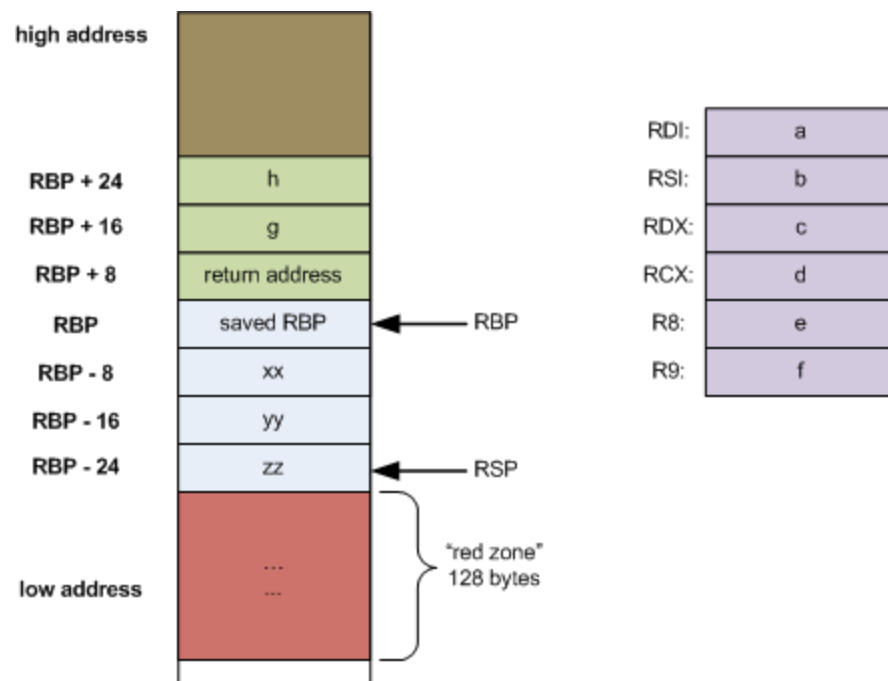
Sistem menggunakan sebuah Stack tunggal untuk menyimpan operand dan hasil sementara.

- **LIFO Principle:** Data dimasukkan (*push*) dan diambil (*pop*) dari puncak stack.
- **Pointers:**
 - **SP (Stack Pointer):** Menunjuk ke alamat elemen teratas di stack.
 - **BP (Base Pointer):** Menunjuk ke alamat awal frame fungsi (untuk akses variabel lokal).
- **Akses Variabel:** Variabel diakses menggunakan alamat relatif terhadap BP (Base Pointer) atau alamat absolut untuk variabel global.



Gambar 1. Mekanisme Operasi Stack (Contoh: Instruksi ADD)

(Sumber: https://users.ece.cmu.edu/~koopman/stack_computers/sec3_2.html)



Gambar 2. Struktur Stack Frame (SP dan BP)

(Sumber: <https://eli.thegreenplace.net/2011/09/06/stack-frame-layout-on-x86-64>)

II. Katalog Instruksi Intermediate Code

Setiap instruksi terdiri dari **Mnemonic** dan optional **Operand**.

Mnemonic	Nama Operasi	Deskripsi Teknis	Perubahan Stack
LIT v	Load Literal	Memasukkan nilai konstanta v ke stack.	push(v)
LOD a	Load Value	Mengambil nilai dari memori alamat a ke stack.	push(mem[a])
STO a	Store Value	Mengambil nilai teratas stack dan simpan di alamat a .	pop(val); mem[a] = val
ADD	Addition	Menjumlahkan dua nilai teratas.	v2=pop; v1=pop; push(v1+v2)
SUB	Subtraction	Mengurangi nilai kedua dengan nilai teratas.	v2=pop; v1=pop; push(v1-v2)
MUL	Multiplication	Perkalian dua nilai teratas.	v2=pop; v1=pop; push(v1*v2)
DIV	Division	Pembagian (integer) dua nilai teratas.	v2=pop; v1=pop; push(v1/v2)
JMP I	Uncond. Jump	Lompat ke label instruksi I .	PC = I

JPC I	Cond. Jump	Lompat ke label <code>I</code> jika nilai teratas stack adalah 0.	<code>val=pop; if val==0 then PC=I</code>
CAL I	Call	Memanggil prosedur/fungsi pada label <code>I</code> .	Simpan konteks (PC, BP)
RET	Return	Kembali dari prosedur ke pemanggil.	Pulihkan konteks

III. Skema Validasi Runtime

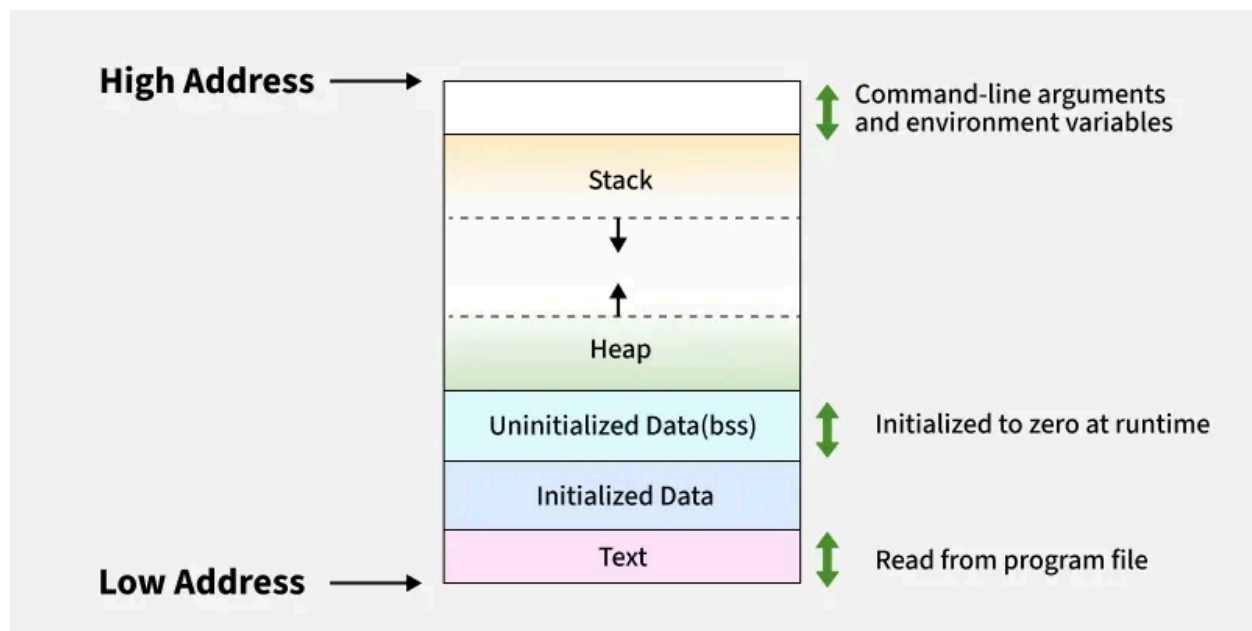
Sebagai bagian dari spesifikasi teknis, Intermediate Code harus berinteraksi dengan **Decorated AST** untuk memastikan validitas sebelum eksekusi dimulai.

Kondisi Error	Mekanisme Deteksi	Tindakan Sistem
Type Mismatch	Menggunakan atribut tipe pada Decorated AST.	Hentikan kompilasi, tampilkan pesan error tipe.
Stack Overflow	Cek <code>SP</code> terhadap batas atas memori stack.	Runtime Error: <code>Stack Overflow</code> .
Stack Underflow	Cek jika instruksi <code>pop</code> dipanggil saat stack kosong.	Runtime Error: <code>Stack Underflow</code> .
Invalid Jump	Validasi keberadaan label sebelum eksekusi.	Runtime Error: <code>Label not found</code> .
OOB Access	Cek <code>address</code> pada <code>LOD/STO</code> terhadap batas memori.	Runtime Error: <code>Memory Access Out of Bounds</code> .

IV. Contoh Penulisan Kode

Berikut adalah contoh bagaimana kode sumber ditransformasikan menjadi urutan Intermediate Code:

Ekspresi:	$y = (3 + 4) * x$
Urutan Intermediate Code:	
LIT 3	[3]
LIT 4	[3, 4]
ADD	[7]
LOD x	[7, val_x]
MUL	[7 * val_x]
STO y	[] (Hasil disimpan ke y)



Gambar 3. Tata Letak Memori (Memory Layout)

(Sumber: <https://www.geeksforgeeks.org/c/memory-layout-of-c-program/>)